Back to Top

No.	Project Setup
	virtual environment:- Create
	virtual environment:- Activate
	Install Fastapi
	upgratde Pip version
	Install Uvicorn
	Create main.py file with route
	Run the server
	Project setup one pc to another Pc

No.	FastAPI Questions
	What is Fastapi
	Fastapi:- Main Features
	FAstapi:- Advantages
	FAstapi:- Disadvantages
	FAstapi:- Example
	Describe Fastapi code
	Starlette
	ASGI
	WSGI
	uvicorn
	Gunicorn
	Gunicorn और Uvicorn में Difference
	FastAPI vs Flask

No.	Endpoint Questions
	GET endpoint
	POST endpoint
	PUT vs PATCH

No.	Path Parameter
	Path Parameter

No.	Path Parameter
	define multiple path parameters?
	validate path parameters
	What is Path()

Query Parameter

No.	Query Parameter
	Query Parameter
	Query parameters:- define multiple?
	Query parameters:- restrict values
	What is Query() in FastAPI?
	Path Parameter vs Query Parameter
	Query parameter optional?

Pydantic & Validation

No.	Pydantic & Validation
	what is BaseModel?

Project Setup

🎯 Create virtual environment

- create the folder and open the cmd

```
python -m venv virtual-name
OR
pip install virtualenv  # Install the package.
virtualenv MyFirstApp
MyFirstApp\scripts\activate
```

Activated virtual environment

```
cd virtual-name\Scripts
d:\mohit\virtual-name\Scripts> activate
(OR)
source env_crud/Scripts/activate
```

For activate

Terminal	Command
PowerShell	`.\virt_env\Scripts\Activate.ps1`
CMD	`virt_env\Scripts\activate.bat`
Git Bash	`source virt_env/Scripts/activate`

🎯 Install Fastapi

- we have install two packages/library **fastapi** and **uvicorn**

```
pip install fastapi
OR
pip install "fastapi[standard]"
```

🎯 upgratde Pip version

```
python.exe -m pip install --upgrade pip
```

🎯 Install Uvicorn

- FastAPI doesn't come with any built-in server application. To run FastAPI app, you need an ASGI server called uvicorn, so install the same too, using pip installer.

```
# You will also need an ASGI server, for production such as Uvicorn or Hypercorn.
pip install "uvicorn[standard]"
```

Uvicorn version

```
uvicorn --version
```

Upgrade Uvicorn

```
pip install --upgrade uvicorn fastapi
```

Uninstall Uvicorn

```
pip uninstall uvicorn fastapi
```

🎯 Create a main.py file with routes

- firstaly active the virtual env
- Create main.py file in folder

Basic Request

```
from fastapi import FastAPI

app = FastAPI()

# Handles GET requests
@app.get("/")
def first_page_function():
    return {"msg":"Hello FastAPI🚀"}

# Handles POST requests
@app.post("/post_route")
def post_function():
    return {"msg":"calling post routes🚀"}

# Handles PUT requests
@app.put("/put_route")
def put_function():
    return {"msg":"calling put routes🚀"}

# Handles PATCH requests
@app.patch("/patch_route")
def patch_function():
    return {"msg":"calling patch routes🚀"}

# Handles DELETE requests
@app.delete("/delete_route")
def delete_function():
    return {"msg":"calling delete routes🚀"}

# Handles HEAD requests
@app.head("/items/{item_id}")
def head_item(item_id: int):
    # This will return the headers without a body
    pass
```

We can use summary, description and tag

```
@app.get("/", summary="Path parameters", description="Path parameters api", tags=
["Path parameters"])
def first_page_function():
    return {"msg":"Hello FastAPI🚀"}
```

🎯 Run the server

- got to directory where main.py exist

```
uvicorn main:app --reload

uvicorn main:app --host 127.8.4.8 --port 12   # Difrent port

# INFO:     Uvicorn running on http://127.0.0.1:8000 (Press CTRL+C to quit)
# INFO:     Started reloader process [28720]
# INFO:     Started server process [28722]
# INFO:     Waiting for application startup.
# INFO:     Application startup complete.
```

```
Open your browser at http://127.0.0.1:8000/
Open your browser at http://127.0.0.1:8000/items
```

Run the server when main.py file in another folder.

```
uvicorn foldername.main:app --reload
```

another way to run server

```
# create run.py file and call main file
import uvicorn

if __name__ == "__main__":
    uvicorn.run("app.main:app", reload=True)

# and run
python run.py
```

🎯 Project setup one pc to another Pc

- First of all we run the command

```
pip freeze > requirements.txt
```

- And other system follows these step

```
# Create and activate virtual environment
virtualenv -p python3 env
. ./env/bin/activate

# Install Python dependencies
pip install -r requirements.txt
pip install --default-timeout=100 -r requirements.txt

# Create SQLite databse, run migrations
cd myapp
./manage.py migrate

# Run Django dev server
./manage.py runserver
```

Fast API Basic Questions

🎯 What is Fastapi

- **FastAPI is a modern, high-performance Python web framework used for building RESTful APIs.**
- It is based on **Starlette** for the web framework and **Pydantic** for data validation.
- It automatically generates interactive API documentation using **OpenAPI (Swagger UI)**.
- Hindi: FastAPI ek modern Python web framework hai jo **API (Application Programming Interface)** banane ke liye use hota hai.

Main Features

- High Performance (ASGI + Starlette ki wajah se)
- Automatic API Documentation (/docs aur /redoc)
- Data Validation using Pydantic
- Easy JWT Authentication support
- Async programming support (async def)

Fastapi Advantages

Advantage	Explanation
High Performance	ASGI aur Starlette ki wajah se bahut fast hai.
Automatic API Docs	`/docs` par Swagger UI automatically milta hai.
Data Validation	Pydantic automatically request data validate karta hai.
Async Support	`async/await` support karta hai, isliye concurrent requests handle kar sakta hai.
Easy to Learn	Flask jaisa simple syntax hai.
Type Hint Support	Python type hints se IDE auto-completion aur fewer bugs milte hain.
OpenAPI Support	Client SDK aur API testing tools ke saath easily integrate hota hai.

Fastapi Disadvantages

Disadvantage	Explanation
Smaller Ecosystem	Django ke comparison me packages aur plugins kam hain.
Built-in Admin Panel Nahi	Django ki tarah ready-made admin interface nahi milta.
Authentication Setup Manual Ho Sakta Hai	JWT, OAuth, permissions wagaira khud configure karne padte hain.
Async Concepts Thode Difficult	Beginners ko `async`, `await`, event loop samajhne me time lag sakta hai.

Disadvantage	Explanation
Large Monolithic Apps Ke Liye Extra Structure Chahiye	Bahut bade projects me architecture manually organize karna padta hai.
Rapid Changes	Kuch libraries aur versions frequently update hote rehte hain.

Simple Example

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def home():
    return {"message": "Hello FastAPI"}

# Run server
uvicorn main:app --reload

# Output:-
{
  "message": "Hello FastAPI"
}
```

Describe Fastapi code

```
from fastapi import FastAPI # This imports the **FastAPI** class which is used to create a web application instance.

app = FastAPI() # This creates an application instance of FastAPI.

@app.get("/")   # decorator is a function that modifies another function. “When someone calls / with GET request, execute this function.”
def home():
    return {"message": "Hello World"} # FastAPI automatically converts Python dictionary into JSON response.
```

🎯 Starlette

- Starlette is a lightweight **ASGI web framework and toolkit** for building asynchronous web applications in Python.
- FastAPI Starlette ke upar built hai.
- Hindi:- Starlette ek lightweight ASGI web framework hai, jiske upar FastAPI bana hua hai.
- It uses Starlette internally for features such as:
    - Routing
    - Request handling
    - Response handling
    - Middleware
    - Background tasks
    - WebSockets
    - Static files
    - Session support

Example

```
from fastapi import FastAPI
from starlette.responses import JSONResponse

app = FastAPI()

@app.get("/")
async def home():
    return JSONResponse({"message": "Hello"})

# Here, JSONResponse comes from Starlette.
```

- Car chalti hai engine ki wajah se, lekin car me extra features hote hain. Usi tarah FastAPI, Starlette ke upar extra features provide karta hai.

🎯 ASGI

- ASGI allows a Python web app (FastAPI, Starlette, Django async, etc.) to **communicate** with an asynchronous server (Uvicorn, Hypercorn, Daphne) and handle many requests concurrently using async/await.
- ASGI (Asynchronous Server Gateway Interface) is a specification that allows Python web applications (FastAPI, Starlette, Django async, etc.) to **communicate** with asynchronous servers such as Uvicorn, Hypercorn, and Daphne. It supports **async/await**, **concurrent requests**, **WebSockets**, and **long-lived connections**.

ASGI Application Signature

- ASGI app ka basic structure:

```
async def app(scope, receive, send):
    pass
```

Parameter	Purpose
scope	Request information (path, method, headers, etc.)
receive	Incoming messages receive karta hai
send	Response bhejta hai

ASGI is Used?

- Handle Multiple requests concurrently
- Support WebSockets
- Support Long-lived connections
- Perform asynchronous operations using async/await

ASGI server

ASGI Server	Use
Uvicorn	FastAPI में सबसे ज्यादा इस्तेमाल होता है
Hypercorn	HTTP/2 और QUIC जैसी features support करता है
Daphne	Django Channels के साथ popular है
Granian	High-performance modern ASGI server
Gunicorn + Uvicorn Workers	Production deployment में बहुत common combination

What is WSGI?

- WSGI (Web Server Gateway Interface) is a standard interface between Python web applications and web servers. Frameworks like Flask and Django use WSGI to communicate with servers such as Gunicorn and uWSGI. WSGI is synchronous and handles one request per worker at a time.
- WSGI (Web Server Gateway Interface) Python web applications aur web servers ke beech communication ka standard interface hai.
- **HINDI:** WSGI web server (Apache, Nginx, Gunicorn) aur Python application (Flask, Django) ke beech bridge ka kaam karta hai.

WSGI ki Limitation

- Agar ek request ko 5 seconds lagte hain, to worker us request ke complete hone tak busy rahega.

🎯 uvicorn

- Uvicorn is an ASGI server used to run FastAPI applications.
- Uvicorn is a lightweight, high-performance ASGI server used to run FastAPI applications.
- Uvicorn एक ASGI server है जो FastAPI application को run करने के लिए इस्तेमाल किया जाता है।

What does Uvicorn do?

- HTTP requests को receive करता है
- ASGI protocol के अनुसार FastAPI से communicate करता है
- Response वापस client को भेजता है
- Async requests को efficiently handle करता है
- WebSocket support भी देता है

Note

- ASGI = एक protocol / specification है (rules/नियम)
- Uvicorn = उन नियमों पर काम करने वाला server
- FastAPI = application/ASGI-compatible framework

🎯 Gunicorn

- Gunicorn (Green Unicorn) is a Python **WSGI HTTP server** used to run Python web applications such as Django and Flask in production environments.
- It works by creating a master process and multiple worker processes to handle concurrent HTTP requests efficiently. For ASGI frameworks like FastAPI, Gunicorn is commonly used together with Uvicorn workers.
- **Hindi:-** यह एक Python WSGI HTTP Server है जिसका उपयोग Python web applications (जैसे Django, Flask) को production में चलाने के लिए किया जाता है।

Gunicorn कैसे काम करता है?

- Gunicorn एक master process बनाता है और उसके अंदर कई worker processes चलाता है।

```
Browser
   |
HTTP Request
   |
Gunicorn (Master)
   |
-----------------
|       |       |
Worker1 Worker2 Worker3
   |
Python App
```

- इससे एक साथ कई requests handle हो सकती हैं।

FastAPI में Gunicorn क्यों?

- Production में जब ज़्यादा traffic हो, तब केवल एक Uvicorn process कम पड़ सकता है।
- इसलिए हम Gunicorn को **process manager** की तरह उपयोग करते हैं।
- Process Manager का मतलब है ऐसा software जो application के processes को manage करता है।

```
gunicorn -w 4 -k uvicorn.workers.UvicornWorker app.main:app
```

Back to Top

🎯 Gunicorn और Uvicorn में Difference

Feature	Gunicorn	Uvicorn
Interface	WSGI	ASGI
Frameworks	Django, Flask	FastAPI, Starlette
Async support	नहीं (native)	हाँ

Feature	Gunicorn	Uvicorn
Use case	Traditional Python web apps	Modern async apps

🎯 FastAPI vs Flask

- Flask is a lightweight web framework, while FastAPI is a modern high-performance API framework with built-in validation, documentation, and async support. | Feature | FastAPI | Flask | | ---------- | ---------- | ---------- | | Release Year | 2018 | 2010 | | Performance | Very Fast (ASGI) | Slower than FastAPI (WSGI) | | Async Support | Built-in (`async/await`) | Limited, requires extra setup | | Data Validation | Automatic with Pydantic | Manual validation | | API Documentation | Auto-generates Swagger & ReDoc | Requires extra libraries | | Type Hints | Strongly uses Python type hints | Optional | | Learning Curve | Slightly higher | Easier for beginners | | Best For | REST APIs, Microservices | Small to Medium Web Apps | | Dependency Injection | Built-in | Not available by default | | WebSocket Support | Built-in | Requires extensions |

Example

- Flask

```
from flask import Flask

app = Flask(__name__)

@app.route("/hello")
def hello():
    return {"message": "Hello World"}
```

- Fastapi

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/hello")
def hello():
    return {"message": "Hello World"}
```

Endpoint Questions

🎯 What is GET endpoint?

- In FastAPI, a GET endpoint is created using the **@app.get() decorator**. The decorator defines the URL path, and the function below it handles the incoming GET request and returns the response, usually as a Python dictionary which FastAPI automatically converts to JSON.
- Hindi:- FastAPI में GET endpoint बनाने के लिए @app.get() decorator का उपयोग किया जाता है।

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Hello FastAPI"}

# Output:-
{
  "message": "Hello FastAPI"
}
```

🎯 POST endpoint

- In FastAPI, a POST endpoint is created using the **@app.post() decorator**.
- We usually define a Pydantic model to validate the request body, and FastAPI automatically converts the incoming JSON into a Python object.

```
@app.post("/items")
def create_item():
    return {"message": "Item created successfully"}
```

🎯 PUT vs PATCH

- PUT → Entire resource ko update karta hai.
- PATCH → Sirf specified fields ko update karta hai.

Path Parameter Questions

🎯 Path Parameter

- A Path Parameter is a **value that is passed as part of the URL path**.
- It is used to **identify a specific resource**.
- Path Parameter is a variable that is included in the URL path and it is used to **identify a specific resource**. In FastAPI, path parameters are defined using curly braces {} in the route.
- **Hindi:-** Path Parameter URL ke andar pass kiya jata hai aur kisi specific record ko identify karne ke liye use hota hai.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(user_id: int):
    return {"user_id": user_id}

# Request
GET /users/101

# Response
{
    "user_id": 101
}
```

🎯 How do you define multiple path parameters?

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users/{user_id}/posts/{post_id}")
def get_post(user_id: int, post_id: int):
    return {
        "user_id": user_id,
        "post_id": post_id
    }
```

How do you validate path parameters?

- In FastAPI, path parameters can be validated using **Path()** with constraints such as **ge**, **gt**, **le**, and **lt**.

```
from fastapi import FastAPI, Path

app = FastAPI()

@app.get("/users/{user_id}")
def get_user(
    user_id: int = Path(..., ge=1, le=1000) # ... means the parameter is required.
):
    return {"user_id": user_id}
```

What is Path()?

- Path() is a FastAPI function used to validate and add metadata to path parameters.
- It allows you to apply rules such as:
    - ge → Greater than or equal to
    - gt → Greater than
    - le → Less than or equal to
    - lt → Less than
- **...** means the parameter is required.
- **Hindi:-** Path() ka use path parameter validation ke liye hota hai.

```
user_id: int = Path(..., ge=1)
```

Query Parameter Questions

🎯 Query Parameter?

- A Query Parameter is a value that is passed in the URL after the ? symbol. It is commonly used for **filtering**, **searching**, **sorting**, and **pagination**.
- Query Parameter URL mein ? ke baad aata hai aur data ko filter ya search karne ke liye use hota hai.

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/products")
def get_products(category: str = None):
    return {"category": category}

# request
GET /products?category=electronics
```

How do you define multiple query parameters?

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users")
def get_users(page: int = 1, limit: int = 10, name: str | None = None):
    return {
        "page": page,
        "limit": limit,
        "name": name
    }

# request :- GET /users?page=2&limit=5&name=Mohit
```

How do you restrict query parameter values?

- In FastAPI, you can restrict query parameter values using **Query validations** such as **min_length**, **max_length**, **ge**, **le**, and **pattern**.
- **Example:-**
- Restrict String Length

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/users")
def get_users(name: str = Query(..., min_length=3, max_length=20)):
    return {"name": name}
```

- Restrict Numeric Range

```
from fastapi import FastAPI, Query

app = FastAPI()

@app.get("/items")
def get_items(page: int = Query(1, ge=1, le=100)):
    return {"page": page}
```

- Restrict to Specific Values

```
from typing import Literal

@app.get("/products")
def get_products(category: Literal["mobile", "laptop", "tablet"]):
    return {"category": category}

# Only these values are allowed: mobile, laptop, tablet
```

What is Query() in FastAPI?

- Query() is a FastAPI function used to validate and configure query parameters.
- It allows you to add validation rules such as:
    - ge → Greater than or equal to
    - le → Less than or equal to
    - min_length → Minimum string length
    - max_length → Maximum string length
    - pattern → Regex validation

```
@app.get("/users")
def get_users(
    page: int = Query(1, ge=1, le=100)
):
    return {"page": page}
```

- **Pehla 1 default value hai.** Matlab agar user page pass nahi karta, to FastAPI automatically page = 1 le lega.
- In Query(1, ge=1, le=100), the **first 1 is the default value.** If the client does not provide the query parameter, FastAPI uses 1 automatically.
- **String Validation Example**

```
@app.get("/search")
def search(
    name: str = Query(..., min_length=3, max_length=20)
):
    return {"name": name}
```

🎯 Path Parameter vs Query Parameter

Path Parameter	Query Parameter
`/users/101`	`/users?id=101`
Specific resource identify karta hai	Filter/search ke liye use hota hai
Required hota hai	Usually optional hota hai

🎯 How do query parameter optional?

- In FastAPI, you can make a parameter optional by giving it a default value, commonly None, and using **Optional** or **| None**.
- **HIndi:-** Parameter ko optional banane ke liye uski default value None set kar dete hain.

Using None

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/users")
def get_users(name: str | None = None):
    return {"name": name}
```

Using Optional

```
from typing import Optional

@app.get("/users")
def get_users(name: Optional[str] = None):
    return {"name": name}
```

🎯 What is a Request Body?

- A Request Body is the data sent by the client to the server in an HTTP request. It is commonly used with POST, PUT, and PATCH requests to send data such as user details, product information, etc.
- **Hindi:-** Request Body woh data hota hai jo client server ko bhejta hai.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    email: str

@app.post("/users")
def create_user(user: User):
    return user

# request
{
    "name": "Mohit",
    "email": "mohit@example.com"
}

# reponse
{
    "name": "Mohit",
    "email": "mohit@example.com"
}
```

Pydantic

Pydantic

- Pydantic is a Python library **used for data validation**, parsing, and serialization using Python type hints.
- FastAPI **uses** Pydantic models to:
  - validate incoming request data,
  - convert data into Python objects,
  - and generate JSON responses automatically.
- Pydantic is a Python library that validates and parses data using type annotations. In FastAPI, it is mainly used to define request and response schemas.

Example

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    return user

# Request
{
  "name": "Mohit",
  "age": 25
}

# Response
{
  "name": "Mohit",
  "age": 25
}
```

- If the client sends wrong value then showing error:

```
# Request

{
  "name": "Mohit",
  "age": "abc"
}

# response
{
  "detail": [
    {
      "loc": ["body", "age"],
      "msg": "Input should be a valid integer",
      "type": "int_parsing"
    }
  ]
}
```

How do you make a field optional?

- To make a field optional in Pydantic, we use Optional[type] from the typing module and assign a default value of None.
- Hindi:- Pydantic में किसी field को optional बनाने के लिए Optional और default value None का उपयोग किया जाता है। इससे वह field request में भेजना आवश्यक नहीं रहता।

```
from typing import Optional
from pydantic import BaseModel

class User(BaseModel):
    name: str
    phone: Optional[str] = None
    phone: str | None = None    # Python 3.10+ Shortcut
```

What is a Pydantic Model?

- A Pydantic model is a Python class that inherits from BaseModel and is used to define the structure, validation rules, and data types for data.
- FastAPI में इसे request body, response body, और data validation के लिए use किया जाता है।

```
from pydantic import BaseModel

class User(BaseModel):
    name: str   # string होना चाहिए
    age: int    # integer होना चाहिए
```

- यह पूरा User class ही Pydantic model कहलाता है।

🎯 What is BaseModel?

- BaseModel Pydantic की मुख्य (core) class है।
- BaseModel is the base class provided by Pydantic that enables automatic data validation, parsing, and serialization using Python type hints.
- जब हम कोई class BaseModel को inherit करके बनाते हैं, तो वह class Pydantic model बन जाती है और उसमें automatic:
    - data validation
    - type checking
    - type conversion
    - JSON serialization
    - error handling

How do you validate request data?

```
from pydantic import BaseModel, Field, EmailStr

class User(BaseModel):
    name: str = Field(min_length=3, max_length=50)
    age: int = Field(gt=0, lt=120)
    email: EmailStr # Email validation
    username: str = Field(min_length=3, max_length=20)
```

- String validation

```
from pydantic import BaseModel, Field

class User(BaseModel):
    username: str = Field(min_length=3, max_length=20)
```

- validate numeric values

```
from pydantic import BaseModel, Field

class Product(BaseModel):
    quantity: int = Field(gt=0, le=100) # Integer Validation
    price: float = Field(gt=0, lt=10000)    # float validation
```

Custom Validation

```
from pydantic import BaseModel, field_validator

class User(BaseModel):
    name: str

    @field_validator("name")
    @classmethod
    def validate_name(cls, value):
        if not value.isalpha():
            raise ValueError("Name should contain only letters")
        return value
```

How do you define default values in Pydantic?

- In Pydantic, default values are defined by assigning a value to the **field directly** or by **using Field(default=value)**. If the client does not provide that field, Pydantic automatically uses the default value.

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    active: bool = True
    age: int = 18
```

- Using Field()

```
from pydantic import BaseModel, Field

class User(BaseModel):
    name: str
    age: int = Field(default=18, gt=0, lt=120)
```

- Optional Field with Default

```
from typing import Optional

class User(BaseModel):
    city: Optional[str] = "Delhi"
```

How do you validate email?

- In Pydantic, email validation is done using the **EmailStr** type. When a field is declared as EmailStr, Pydantic automatically checks whether the provided value is a valid email address and raises a validation error if the format is incorrect.
- Hindi:- FastAPI / Pydantic में email validate करने के लिए EmailStr का उपयोग किया जाता है।
- EmailStr use करने के लिए package install होना चाहिए:

```
pip install email-validator
```

- Example

```
from pydantic import BaseModel, EmailStr

class User(BaseModel):
    name: str
    email: EmailStr # Email validation
    optional_email: Optional[EmailStr] = None   # optional email
```

What is Field()?

- Field() is a Pydantic function used to define additional validation rules, default values, and metadata for model fields. It allows us to apply constraints such as min_length, max_length, gt, lt, and also provide descriptions and examples for FastAPI documentation.
- Hindi:- Field() Pydantic का helper function है जिसका उपयोग model की fields पर extra validation, default values, constraints और Swagger documentation metadata लगाने के लिए किया जाता है। इससे हम min_length, max_length, gt, ge जैसी validations आसानी से define कर सकते हैं।

What is Pydantic serialization?

- Pydantic serialization is the process of **converting** a **Pydantic model (Python object)** into a **JSON-compatible** format such as a dictionary or JSON string. It is commonly done **using model_dump()** and **model_dump_json()**. FastAPI automatically uses Pydantic serialization when returning API responses.
- Hindi:- Pydantic serialization वह प्रक्रिया है जिसमें Pydantic model (Python object) को JSON-compatible format जैसे dictionary या JSON string में convert किया जाता है। इसके लिए model_dump() और model_dump_json() methods का उपयोग किया जाता है। FastAPI response भेजते समय इसे automatically use करता है।
- Pydantic models internally Python objects होते हैं, लेकिन API response भेजने के लिए उन्हें JSON में बदलना पड़ता है। यही process Pydantic serialization कहलाती है।

Python Object	JSON Response
User(name="Mohit", age=25)	{"name":"Mohit","age":25}

1. model_dump():-

2. model_dump_json():-

model_dump():-

- model_dump() is a Pydantic v2 method used to convert a Pydantic model into a Python dictionary.
- model_dump() Pydantic v2 का method है जो Pydantic model को Python dictionary में convert करता है। यह serialization के लिए उपयोग किया जाता है और Pydantic v1 के dict() method का replacement है। इसका उपयोग API response, database save, और data processing में किया जाता है।

1. simple example Real life example

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

user = User(name="Mohit", age=25)

print(user.model_dump())

# Output:-
{
  "name": "Mohit",
  "age": 25
}
```

Why is it Used?

- जब हमें:
  - database में save करना हो,
  - custom JSON response बनाना हो,
  - logging करनी हो,
  - या data को manipulate करना हो,
- तब model_dump() उपयोग करते हैं।

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class User(BaseModel):
    name: str
    age: int

@app.post("/users")
def create_user(user: User):
    data = user.model_dump()
    return {
```

```
        "message": "User created",
        "data": data
    }
```

2. Exclude a Field

```
user.model_dump(exclude={"age"})

# Output:-
{
  "name": "Mohit"
}
```

3. user.model_dump(include={"name"})

```

```

What is model_validate() in Pydantic?

- model_validate() is used to validate data and create a Pydantic model instance from a dictionary or other input data.
- In Pydantic v2, model_validate() replaces many uses of parse_obj() from v1.
- **HINDI:-** model_validate() ka use dictionary ya input data ko validate karke Pydantic object banane ke liye hota hai.

```
from pydantic import BaseModel

class User(BaseModel):
    name: str
    age: int

data = {
    "name": "Mohit",
    "age": 25
}

user = User.model_validate(data)

print(user) # Output:- name='Mohit' age=25
```

Difference: model_validate() vs model_dump()

Method	Purpose
`model_validate()`	Dict → Pydantic Model

Method	Purpose
`model_dump()`	Pydantic Model → Dict

What is Pydantic deserialization?

- Pydantic deserialization is the process of converting incoming JSON or dictionary data into a Pydantic model (Python object). During this process, Pydantic automatically validates the data, performs type conversion, and raises validation errors if the input is invalid. FastAPI uses Pydantic deserialization automatically for request bodies.
- Pydantic deserialization वह प्रक्रिया है जिसमें incoming JSON या dictionary data को Pydantic model (Python object) में convert किया जाता है। इस दौरान Pydantic automatically data validation और type conversion करता है। FastAPI request body को handle करने के लिए इसी deserialization process का उपयोग करता है।
- Deserialization is the process of **converting JSON or dictionary data** into a **Pydantic model (Python object)**.
- Hindi:- जब client API को JSON भेजता है, तो Pydantic उस JSON को Python object में बदल देता है। इसी को Pydantic deserialization कहते हैं।

How do you create nested Pydantic models?

- A nested Pydantic model means **one Pydantic model is used inside another model**.
- Nested Pydantic models are created by using one Pydantic model as a field inside another model. This helps validate complex and hierarchical JSON data.
- Agar ek model ke andar doosra model use kiya jaye, to use Nested Pydantic Model kehte hain.

```
from fastapi import FastAPI
from pydantic import BaseModel

app = FastAPI()

class Address(BaseModel):
    city: str
    state: str

class User(BaseModel):  # yaha eke model ke andar dusra model call kara hai
    name: str
    age: int
    address: Address

@app.post("/users")
def create_user(user: User):
    return user

    # How to call
    user = User.model_validate(data)
    print(user.address.city)   # Noida
```

Why Use Nested Models?

- Organize complex data
- Better validation
- Cleaner API schema
- Reusable models

How do you validate nested objects?

- **Nested objects** are **validated** by defining **nested Pydantic models**. Pydantic automatically validates all nested fields and raises a validation error if any required field is missing or has the wrong type.
- Pydantic automatically check karega ki address ke andar city aur state sahi hain ya nahi.

What is JSONResponse?

- JSONResponse is a FastAPI/Starlette response class used when you want to manually control the JSON response, including the status code, headers, and response content.
- **HINDI:-** JSONResponse ka use tab karte hain jab humein API ka JSON response manually control karna ho.

```
from fastapi import FastAPI
from fastapi.responses import JSONResponse

app = FastAPI()

@app.get("/users")
def get_users():
    return JSONResponse(
        content={
            "status": True,
            "message": "Users fetched successfully"
        },
        status_code=200
    )
```

What is ORJSONResponse?

- ORJSONResponse is a FastAPI response class that uses the orjson library to serialize Python data into JSON.
- It is mainly used when you want faster JSON serialization than the standard JSON encoder.

```
from fastapi import FastAPI
from fastapi.responses import ORJSONResponse

app = FastAPI()

@app.get("/users")
def get_users():
    return ORJSONResponse(
        content={
            "status": True,
            "message": "Users fetched successfully"
        },
        status_code=200
    )
```

- **NOTE:-** JSONResponse & ORJSONResponse same hi hai, ORJSONResponse tab use karte hai jab large data aata ho api se mtb bada data ho

How do you connect MySQL with FastAPI?

- I connect FastAPI to MySQL using **SQLAlchemy ORM** with a MySQL driver such as **PyMySQL**, create an engine and session factory, and inject the database session into API routes using FastAPI's Depends().

Steps

```
# 1. Install packages
pip install fastapi uvicorn sqlalchemy pymysql

# 2. Create database URL
DATABASE_URL = "mysql+pymysql://root:password@localhost:3306/fastapi_db"

# 3. Create SQLAlchemy engine
from sqlalchemy import create_engine

engine = create_engine(DATABASE_URL)

# 4. Create session
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

# 5. Create FastAPI database dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# 6. Use it in an API
from fastapi import Depends, FastAPI
from sqlalchemy.orm import Session

app = FastAPI()

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    users = db.query(User).all()
    return users
```

How do you connect PostgreSQL with FastAPI?

- I connect PostgreSQL with FastAPI using SQLAlchemy and the Psycopg2/Psycopg driver, create an engine and session, and inject the database session into routes using FastAPI's dependency injection.

Steps

```
# 1. Install packages
pip install fastapi uvicorn sqlalchemy psycopg2-binary

# 2. Create database URL
DATABASE_URL = "postgresql://postgres:password@localhost:5432/mydb"

# 3. Create SQLAlchemy engine
from sqlalchemy import create_engine

engine = create_engine(DATABASE_URL)

# 4. Create session
from sqlalchemy.orm import sessionmaker

SessionLocal = sessionmaker(
    autocommit=False,
    autoflush=False,
    bind=engine
)

# 5. Create FastAPI database dependency
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# 6. Use it in an API
from fastapi import FastAPI, Depends
from sqlalchemy.orm import Session

app = FastAPI()

@app.get("/users")
def get_users(db: Session = Depends(get_db)):
    return {"message": "Connected to PostgreSQL"}
```

What is SQLAlchemy?

- SQLAlchemy is a Python ORM and database toolkit that enables developers to interact with databases using Python objects instead of writing raw SQL queries.
- HINDI:- SQLAlchemy ek Python library hai jo database ke saath interact karne ke liye use hoti hai. Yeh ORM provide karti hai, jisse hum SQL queries likhne ke bajay Python classes aur objects ka use karke database operations kar sakte hain.

Why use SQLAlchemy?

- Database operations become easier.
- Reduces the need to write raw SQL queries.
- Supports multiple databases (MySQL, PostgreSQL, SQLite, Oracle, etc.).
- Provides ORM and Core SQL functionality.
- Helps prevent SQL Injection attacks through parameterized queries.

What is a Database Session?

- A database session is a temporary connection between the application and the database that is used to perform database operations such as Create, Read, Update, and Delete (CRUD). It manages transactions and ensures proper communication with the database.

Why do we need a Session?

- Executes database queries.
- Tracks changes to objects.
- Manages transactions (commit, rollback).
- Closes the connection after use.
- Prevents connection leaks.

Session Lifecycle

```
Create Session
      ↓
Execute Queries
      ↓
Commit / Rollback
      ↓
Close Session
```

Alembic

What is Alembic?

- Alembic is a **database migration tool** for SQLAlchemy. It helps manage and version-control database schema changes such as creating tables, adding columns, modifying columns, or deleting tables without manually writing SQL scripts.

Why do we use Alembic?

- Without Alembic:
  - You have to manually run SQL queries to change the database schema.
  - Keeping schema changes synchronized across environments becomes difficult.
- With Alembic:
  - Tracks schema changes as migration files.
  - Supports version control for the database.
  - Makes deployments easier and safer.

Setup/install Alembic

- Steps

```
# Install Alembic
pip install alembic

# Initialize Alembic
alembic init alembic

# IN your folder
project/
│
├── alembic/
│   ├── versions/
│   ├── env.py
│   ├── script.py.mako
│
├── alembic.ini
```

Configure Alembic

- Open **alembic.ini** file
    - Find:- sqlalchemy.url = driver://user:pass@localhost/dbname
    - Change to: sqlalchemy.url = mysql+pymysql://root:password@localhost:3306/fastapi_db
- open **alembic/env.py** file
    - find:- target_metadata = None
    - Replace with:

```
from database.connection import Base
import models

target_metadata = Base.metadata
```

- change one line

```
# before
with connectable.connect() as connection:
    context.configure(
        connection=connection,
        target_metadata=target_metadata,
    )

    with context.begin_transaction():
        context.run_migrations()


# after
with connectable.connect() as connection:
    context.configure(
        connection=connection,
        target_metadata=target_metadata,
        render_as_batch=True   # ✅ add this line
    )

    with context.begin_transaction():
        context.run_migrations()
```

What is alembic.ini?

- alembic.ini is the main configuration file of Alembic. It stores database connection settings and migration-related configurations that Alembic uses when generating and applying migrations.

What is env.py in Alembic?

- env.py is the core Alembic configuration script **that controls how migrations are generated and executed**. It connects Alembic with your SQLAlchemy models and database.
- When you run:
  - alembic revision --autogenerate -m "message"
  - OR - alembic upgrade head
  - Alembic first executes env.py.

Main Responsibilities of env.py

1. Load Database Configuration
2. Connect Alembic to Models
3. Create Database Connection
4. Run Migrations

What is the versions folder in Alembic?

- The versions folder stores all Alembic migration files. Each migration file represents a specific database schema change and allows Alembic to track database versions over time.
- **HINDI:-** versions folder me Alembic ki sari migration files store hoti hain. Har file database schema me kiye gaye ek change ko represent karti hai, jaise table create karna, column add karna ya column remove karna.

```
alembic/
│
├── env.py
├── versions/
│   ├── 1a2b3c4d_initial_migration.py
│   ├── 5e6f7g8h_add_email_column.py
│   └── 9i0j1k2l_create_roles_table.py
```

Why is the versions Folder Important?

- Maintains migration history.
- Enables schema version control.
- Supports rollback (downgrade).
- Keeps development, staging, and production databases in sync.

Example

- Suppose your User model is:

```
class User(Base):
    __tablename__ = "users"

    id = Column(Integer, primary_key=True)
    name = Column(String)
```

- Later you add:-

```
email = Column(String)
```

Common Alembic Commands

- Initialize Alembic

```
alembic init alembic
```

- Create Migration

```
alembic revision --autogenerate -m "add email column"
```

- Apply Migration

```
alembic upgrade head
```

- Roll Back One Version

```
alembic downgrade -1
```

- Check Current Version

```
alembic current
```

Authentication & Authorization

Authentication vs Authorization?

- Authentication verifies the identity of a user (who you are), while Authorization determines the permissions and resources that the authenticated user can access (what you can do).